

A Hybrid Acceleration for Self-Gravity Calculation in Infinitesimally Thin Disks with Adaptive Mesh Refinement

CHIEN-CHANG YEN (嚴健彰) ^{1,2} YUNG-CHING SUN (孫詠晴) ^{2,3} YAO-HUAN TSENG (曾耀寰) ²
HSIEN SHANG (尚賢) ² AND RUBEN KRASNOPOLSKY ⁴

¹*Department of Mathematics, Fu Jen Catholic University, New Taipei City, Taiwan*

²*Institute of Astronomy and Astrophysics, Academia Sinica, Taipei 106216, Taiwan*

³*Department of Mechanical Engineering, National Taiwan University, Taiwan*

⁴*Institute of Astronomy and Astrophysics, Academia Sinica, Taiwan*

ABSTRACT

Tseng et al. (2019) advanced a direct approach for the computation of self-gravitational force to second-order accuracy based on adaptive mesh refinement (AMR). This direct approach is free of artificial boundary conditions, and it is possible to solve astronomical problems of gaseous disks in reality with speedups by graphics processing units (GPUs). On the other hand, the fast Fourier transform (FFT) on uniform grids requires variation in the number of zones. A hybrid **method (HM)** of the direct and the FFT methods can effectively replace the GPU acceleration in speeding up the computation. Numerical simulations demonstrate the computational time of the **HM** is less than that of the direct plus GPU acceleration and scales nearly linearly as the total number of zones when a sufficiently large number of zone sizes is selected. An application to a two-arm spiral is demonstrated for the generality of the proposed method.

Keywords: self-gravity-methods:numerical

1. INTRODUCTION

Self-gravitational force calculation on finite disks is significant in the evolution of disk galaxies (Kim et al. 2012; Lee & Shu 2012; Lee 2014; Lin et al. 2008, 2013). The infinitesimally thin disks are often adopted and approximated by two-dimensional hydrodynamical simulations coupled with a Poisson solver for a first approximation to reduce numerical complexity. James (1977) advanced a Poisson solver for handling isolated boundary conditions involving a Green function with fast Fourier transform (FFT). Yen et al. (2012) developed a direct self-gravitational force calculation of second-order accuracy based on uniform discretization of the Cartesian coordinates without imposing artificial boundary conditions. The formula is a double sum in Yen et al. (2012), and the force calculation has the numerical complexity $O(N^4)$, where N is the number of zones in one dimension if a direct calculation is used. It is similar to the N -body calculation. The double sum is a convolution type, and the fast Fourier transform (FFT) can speed it up. The numerical complexity can be reduced from $O(N^4)$ to $O(N^2 \log^2 N)$ for uniform discretization, and the computational time is reasonable.

The extension of the uniform-mesh direct summation technique to adaptive mesh refinement (AMR) is considered in Tseng et al. (2019), which speeds up the computation when the graphics processing unit (GPU) is utilized. Despite this improvement, the FFT, when deployed, is faster than a direct N -body calculation (DnBC) whenever N is large enough. The discretization is crucial for the representation of a numerical computation as a convolution form for the integral that calculates the force field.

This work improves upon the computational time reported in Tseng et al. (2019) by employing a combined methodology involving the DnBC and FFT techniques, utilizing domain decomposition facilitated by AMR. This approach originated from observing the numerical intricacies of DnBC and FFT computations concerning self-gravitational force,

particularly their implementation on the uniform, nested, and AMR grids. Notably, in this proposed **hybrid** method (**HM**), the FFT specifically contributes to the acceleration of the computational time if the computational zones on AMR could be decomposed into the union sum of sub-regions of uniform zones. Alternative approaches for solving Poisson equations, such as deriving force fields (Tomida et al. 2023; Mandal et al. 2023; Deriaz E. 2023; Maeda et al. 2024), may demand an accuracy higher than second-order to ensure a similarly accurate force field.

The paper is structured as follows: Section 2 provides an overview of the second-order force calculation using adaptive mesh refinement (AMR). Section 3 presents an analysis of the numerical complexities associated with DnBC and FFT, conducted through an experiment employing uniform grid discretization. The hybrid method (**HM**), integrating DnBC and FFT within an AMR framework, is detailed in Section 4, followed by numerical examples in Section 5. Concluding remarks and discussions are presented in the final section.

2. A REVIEW ON FORCE CALCULATION OF SECOND ORDER

This direct method offers enhanced flexibility in grid discretization, simplifying the conceptual framework and facilitating straightforward implementation to ensure a comprehensive initial force calculation using AMR.

We define $K(x, y, z) = \frac{1}{\sqrt{x^2 + y^2 + z^2}}$. The forces are described in Yen et al. (2012) as

$$F_x(x, y, 0) = - \iint \frac{\partial}{\partial x} K(\bar{x} - x, \bar{y} - y, 0) \sigma(\bar{x}, \bar{y}) d\bar{x} d\bar{y}, \quad (1)$$

and

$$F_y(x, y, 0) = - \iint \frac{\partial}{\partial y} K(\bar{x} - x, \bar{y} - y, 0) \sigma(\bar{x}, \bar{y}) d\bar{x} d\bar{y}. \quad (2)$$

For simplicity, we denote the symbols $\Phi(x, y) = \Phi(x, y, 0)$ and $K(x, y) = K(x, y, 0)$ between the two dimensions (2D) and three dimensions (3D). Note that the Green's function is still a kernel of 3 dimensions. We further assume that the surface density distribution $\sigma(x, y)$ vanishes outside of a computational domain (assumed to be a rectangle) Ω in 2D. The domain Ω is split into sub-regions Ω_i that are mutually disjoint except at the boundary and $\Omega = \bigcup_{i=1}^{\mathcal{N}} \Omega_i$. The surface density could be rewritten as

$$\sigma = \sum_{i=1}^{\mathcal{N}} \sigma_i(x, y), \quad (3)$$

where $\sigma_i(x, y) = \sigma(x, y) \chi_{\Omega_i}$, χ_{Ω_i} is a characteristic function of the domain Ω_i ,

$$\chi_{\Omega_i}(x, y) = \begin{cases} 1, & (x, y) \in \Omega_i, \\ 0, & (x, y) \notin \Omega_i. \end{cases} \quad (4)$$

The positive integer $\mathcal{N}$ is the total number of zones generated by grid discretization. From Eqs. (1) and (3), it follows that

$$-\frac{\partial}{\partial x} \Phi(x, y) = \sum_{i=1}^{\mathcal{N}} \iint_{\Omega_i} \frac{\partial}{\partial x} (K(\bar{x} - x, \bar{y} - y)) \sigma_i(\bar{x}, \bar{y}) d\bar{x} d\bar{y}. \quad (5)$$

In Yen et al. (2012), the restricted surface function σ_i on Ω_i is approximated by a Taylor polynomial of degree one, given as

$$\sigma_i(x, y) \approx \sigma_i^0 + \delta_i^x (x - x_i^c) + \delta_i^y (y - y_i^c) \quad (6)$$

in the sub-rectangle region Ω_i , with (x_i^c, y_i^c) being the center of the sub-rectangle Ω_i and the slopes δ_i^x and δ_i^y for x and y directions, respectively. Substituting equations (6) into (5) gives us

$$\begin{aligned}
-\frac{\partial}{\partial x}\Phi(x, y) &\approx -\sum_{i=1}^{\mathcal{N}} \iint_{\Omega_i} \frac{\partial}{\partial x}(K(\bar{x} - x, \bar{y} - y)) [\sigma_i^0 + \delta_i^x(x - x_i^c) + \delta_i^y(y - y_i^c)] d\bar{x} d\bar{y} \\
&= -\sum_{i=1}^{\mathcal{N}} \sigma_i^0 \iint_{\Omega_i} \frac{\partial}{\partial x}(K(\bar{x} - x, \bar{y} - y)) d\bar{x} d\bar{y} \\
&\quad - \sum_{i=1}^{\mathcal{N}} \delta_i^x \iint_{\Omega_i} \frac{\partial}{\partial x}(K(\bar{x} - x, \bar{y} - y))(\bar{x} - x_i^c) d\bar{x} d\bar{y} \\
&\quad - \sum_{i=1}^{\mathcal{N}} \delta_i^y \iint_{\Omega_i} \frac{\partial}{\partial x}(K(\bar{x} - x, \bar{y} - y, 0))(\bar{y} - y_i^c) d\bar{x} d\bar{y}.
\end{aligned} \tag{7}$$

We define $F_i^x(x, y) = -\frac{\partial}{\partial x}\Phi(x, y)$, and obtain

$$F_i^x(x, y) \approx F_i^{x,0}(x, y) + F_i^{x,x}(x, y) + F_i^{x,y}(x, y) \tag{8}$$

where

$$F_i^{x,0}(x, y) = -\sum_{i=1}^{\mathcal{N}} \sigma_i^0 K_i^{x,0}(x, y), \tag{9}$$

$$F_i^{x,x}(x, y) = -\sum_{i=1}^{\mathcal{N}} \delta_i^x K_i^{x,x}(x, y), \tag{10}$$

$$F_i^{x,y}(x, y) = -\sum_{i=1}^{\mathcal{N}} \delta_i^y K_i^{x,y}(x, y), \tag{11}$$

and

$$K_i^{x,0}(x, y) = \iint_{\Omega_i} \frac{(\bar{x} - x)}{((\bar{x} - x)^2 + (\bar{y} - y)^2)^{3/2}} d\bar{x} d\bar{y}, \tag{12}$$

$$K_i^{x,x}(x, y) = \iint_{\Omega_i} \frac{(\bar{x} - x_i^c)(\bar{x} - x)}{((\bar{x} - x)^2 + (\bar{y} - y)^2)^{3/2}} d\bar{x} d\bar{y}, \tag{13}$$

$$K_i^{x,y}(x, y) = \iint_{\Omega_i} \frac{(\bar{y} - y_i^c)(\bar{x} - x)}{((\bar{x} - x)^2 + (\bar{y} - y)^2)^{3/2}} d\bar{x} d\bar{y}. \tag{14}$$

We further decompose the sub-rectangle domain Ω_i as sub-cell $\Omega_i = [x_i^\ell, x_i^r] \times [y_i^d, y_i^u]$ in computation. The full expression of force kernels that are summarized below was formulated in Yen et al. (2012) when $z = 0$, using the intervals $[x_i^\ell, x_i^r]$ and $[y_i^d, y_i^u]$:

$$K_i^{x,0}(x, y) = -\ln((\bar{y} - y) + \sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2}) \Big|_{x_i^\ell}^{x_i^r} \Big|_{y_i^d}^{y_i^u}, \tag{15}$$

$$K_i^{x,x}(x, y) = (x - x_i^c) K_i^{x,0}(x, y) + [(\bar{y} - y) \ln((\bar{x} - x) + \sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2})] \Big|_{x_i^\ell}^{x_i^r} \Big|_{y_i^d}^{y_i^u}, \tag{16}$$

$$K_i^{x,y}(x, y) = (y - y_i^c) K_i^{x,0}(x, y) - [\sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2}] \Big|_{x_i^\ell}^{x_i^r} \Big|_{y_i^d}^{y_i^u}, \tag{17}$$

$$K_i^{y,0}(x, y) = -\ln((\bar{x} - x) + \sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2}) \Big|_{x_i^\ell}^{x_i^r} \Big|_{y_i^d}^{y_i^u}, \tag{18}$$

$$K_i^{y,x}(x, y) = (x - x_i^c) K_i^{y,0}(x, y) - [\sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2}] \Big|_{x_i^\ell}^{x_i^r} \Big|_{y_i^d}^{y_i^u}. \tag{19}$$

$$K_i^{y,y}(x, y) = (y - y_i^c) K_i^{y,0}(x, y) + [(\bar{x} - x) \ln((\bar{y} - y) + \sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2})] \Big|_{x_i^\ell}^{x_i^r} \Big|_{y_i^d}^{y_i^u}. \tag{20}$$

This full expression can be evaluated if the position (x, y) does not coincide with the position $\bar{x} = x$ or $\bar{y} = y$ on the plane, $z = 0$. For uniform grid discretization, the center of the zone, $\bar{x}$, and $\bar{y}$ will never fall on the interface; however,

this situation may not be completely avoidable in the context of nested grid Wang et al. (2016) or the AMR. When such situation occurs, the Kernel $K_i^{x,0}(x, y)$ can be modified to be

$$-\text{sgn}(|\bar{x} - x|) \ln((\bar{y} - y) + \sqrt{(\bar{x} - x)^2 + (\bar{y} - y)^2}) - (1 - \text{sgn}(|\bar{x} - x|)) \text{sgn}(\bar{y} - y) \ln|\bar{y} - y|, \quad (21)$$

which can still be used for the uniform grids, but evaluating it would take more computational time.

3. A COMPARISON STUDY OF DNBC AND FFT ON UNIFORM GRIDS

Let us first construct the grid discretization using AMR and make a table of the structure of zones for AMR. For simplicity, the domain of consideration is confined to a rectangle for the coarsest grid discretized, say $\Omega = [-1, 1] \times [-1, 1]$. Furthermore, each refinement subdivides each parent zone into four sub-rectangles. There are 9 elements as a tuple $(x_c, y_c, L, P, EN, LN, DN, RN, UN)$ to record the information of zones. The center (x_c, y_c) is located at level L for each cell, with parent P and the end of tree nodes or the leaf nodes EN which have no children. The elements LN, DN, RN , and UN , are the nearby nodes on the four sides of left, down, right, and up, respectively. An example of 16 zones at the coarsest level 2 is given in Tseng et al. (2019). This grid generation allows a more straightforward implementation into hydrodynamical simulations. In hydrodynamic simulations, one must consider the grid zones at the same and different levels to calculate flux. Here, we concentrate on the self-gravitational force calculation, which is only restricted to the leaf nodes (zones). We apply our approach to an AMR grid generated in the sense of Berger & Oliger (1984).

The D_2 disk has the following surface density (Shulz (2009)):

$$\Sigma_{D_2}(R; \alpha) = \begin{cases} \sigma_0(1 - R^2/\alpha^2)^{3/2} & \text{for } R \leq \alpha \\ 0 & \text{for } R > \alpha \end{cases} \quad (22)$$

where $R = \sqrt{x^2 + y^2}$ and α is a constant. The corresponding radial force on the plane is

$$F_{R,D_2}(R; \alpha) = \begin{cases} -\frac{3\pi^2\sigma_0RG}{16\alpha^3}(4\alpha^2 - 3R^2) & \text{for } R \leq \alpha, \\ -\frac{2\pi\sigma_0G}{8\alpha^2} \left[R(4\alpha^2 - 3R^2) \sin^{-1}\left(\frac{\alpha}{R}\right) - \alpha(2\alpha^2 - 3R^2) \sqrt{1 - \alpha^2/R^2} \right] & \text{for } R > \alpha. \end{cases} \quad (23)$$

We will use the D_2 disk with $\alpha = 0.25$ shown above and the $D_{2,2}$ disk, which consists of two D_2 disks that have centers located at $(0, 0.5)$ and $(0, -0.5)$ of D_2 , respectively, with $\alpha = 0.25$. Let us denote the $AMR(L, D)$ as the case of demonstration for the level L of AMR and the disk D . Here, the level L is a positive integer and $D = D_2$ or $D_{2,2}$, where $L = 1$ represents the uniform grid discretization. These two approaches for mesh refinement are used in Tseng et al. (2019). The first one is the independent level refinement (ILR), which sets the distance between the edges of the next level as less than 0.05 of the coarser grid for each refinement. The other one is called variable level refinement (VLR), which is to set the distance between the edges of the next grid level less than $0.2 \times d^{1-L}$ for each of the levels $L > 1$. **The value d introduces the flexibility on distance between the edges of the next level as less than d of the coarser grid for each refinement.** More precisely, the mesh refinement criterion for ILR is to fix the ratio of two successive refinements, and for the VLR is to vary the ratio of two successive refinements with d . The distinction of ILR and VLR is that ILR is more accurate than VLR, but the computation of ILR is more expensive than that of VLR.

DnBC is the most straightforward approach and is flexible, but it is expensive. It is known that the order estimation of computational complexity for DnBC is $O(nm)$, where n is the number of zones of the forces and m is the number of zones of the given surface density. The order of FFT is $O(n^2 \log^2 n)$ for two-dimensional surface density, where n is the number of zones in one dimension for a uniform discretization.

We demonstrate a study of numerical computational time for the DnBC and FFT on uniform grids in Table 1 for the disks $D = D_2$ and $D_{2,2}$. We employ two computational platforms: Intel(R) Xeon(R) CPU E5-2698 v4 @ 2.20 GHz and Nvidia Tesla P100. The C++ code runs on a CentOS Linux release 7.6.1810. The code is implemented with FFTW3 v3.3.8 and compiled by Intel compiler version 19.0.2.187. The CUDA code runs on the same machine as the Nvidia Tesla P100. It is collected by Nvidia CUDA Compiler (NVCC) version 11.3. We calculate the $AMR(1, D_2)$ in Tseng et al. (2019) for D_2 and $D_{2,2}$ with ILR mesh refinement. More precisely, the DnBC is done by CUDA on GPU while the FFT is done by FFTW on CPU. The results of the computational time are shown in Table 1 for the number of zones in one dimension is $N = 2^K$, where $K = 7, 8, \dots, 11$. The computational time CT(DnBC) and CT(FFT) for

DnBC and FFT approaches are given in the second and third rows, respectively. The computational time is calculated by averaging 30 runs for each case. These results show that the FFT is faster than the DnBC by about 17 or 121 times, which motivates us to implement the FFT in the computation if it can be used. The initial computation time is dominated by startup cost, which underlies the drop in the ratios shown for $K = 7 - 9$.

K	7	8	9	10	11
CT(DnBC)- D_2	4.29	7.28	15.60	171.0	2630
CT(FFT)- D_2	0.10	0.25	0.90	4.49	21.71
CT(DnBC)- D_2 /CT(FFT)- D_2	43.33	29.04	17.35	38.08	121.14
CT(DnBC)- $D_{2,2}$	4.39	5.29	15.7	171.0	2620
CT(FFT)- $D_{2,2}$	0.10	0.24	0.90	4.41	21.62
CT(DnBC)- $D_{2,2}$ /CT(FFT)- $D_{2,2}$	44.19	22.10	17.38	38.78	121.17

Table 1. Computing time of self-gravitational force using the formulation in Section 3 for the direct N -body calculation (DnBC) and fast Fourier transform (FFT) based on uniform discretization of Cartesian grids. The first row shows the number of zones $N = 2^K$, $K = 7, 8, \dots, 11$ in one dimension. The computational time $CT(DnBC)$ and $CT(FFT)$ for the DnBC and FFT approaches for D_2 are given in the second and third rows, respectively. The ratio $CT(DnBC)/CT(FFT)$ is given in the fourth row. Similarly, $CT(DnBC)$ and $CT(FFT)$ for the DnBC and FFT approaches for $D_{2,2}$ are given in the bottom table. The results show that the FFT is faster than the DnBC about 17 or 121 times.

4. A HYBRID APPROACH ACCELERATION

In this section, we detail the combined approach, **hybrid method (HM)**, utilizing DnBC and FFT for computing self-gravitational force in infinitesimally thin disks with the support of AMR. While the direct method's implementation is straightforward, the FFT part is divided into two cases: one where the force is calculated on coarser grids while the density is provided on finer grids, and the reverse scenario.

Let us start with the numerical computational domain considered here $\Omega = [-1, 1] \times [-1, 1]$. For each positive integer K , the domain Ω is divided into $2^K \times 2^K = 2^{2K}$ sub-regions or zones, $D_{i,j}^K = [x_{i-1}^K, x_i^K] \times [y_{j-1}^K, y_j^K]$, where $x_i^K = -1 + \frac{2i}{2^K}$ and $y_j^K = -1 + \frac{2j}{2^K}$ for $i = 1, 2, \dots, 2^K$ and $j = 1, 2, \dots, 2^K$. The center of each zone $D_{i,j}^K$ is denoted by $(x_{i-1/2}^K, y_{j-1/2}^K)$ with $x_{i-1/2}^K = -1 + \frac{2i-1}{2^K}$ and $y_{j-1/2}^K = -1 + \frac{2j-1}{2^K}$. An AMR structure is a collection of sets S^ℓ which collects several regions $D_{i,j}^\ell$ for some index i and j for $\ell = K_C, K_C + 1, \dots, K_F$ such that the intersection of the S^ℓ

and $S^{\tilde{\ell}}$ are only on the edges if $\ell \neq \tilde{\ell}$ and the union $\bigcup_{\ell=K_C}^{K_F} S^\ell = \Omega$. The force calculation computes the force at the center of all zones in S^ℓ , where $\ell = K_C, K_C + 1, \dots, K_F$.

We suppose the formula for the x -force F_x at the center $(x_{i-1/2}^\ell, y_{j-1/2}^\ell)$ due to the surface density $\sigma_{i',j'}^{\ell'}$ is given in the center of the zone $D_{i',j'}^{\ell'}$ can be represented by the first-order approximation $\sigma_{i',j'}^{\ell'} \mathcal{F}_0(x_{i'-1/2}^{\ell'} - x_{i-1/2}^\ell, y_{j'-1/2}^{\ell'} - y_{j-1/2}^\ell)$ using the constant surface density approximation on each cell. The first-order approximation is shown in Tseng et al. (2019) as

$$\mathcal{F}_0 = -\ln \left[(\bar{y} - y_{j-1/2}^\ell) + \sqrt{(\bar{x} - x_{i-1/2}^\ell)^2 + (\bar{y} - y_{j-1/2}^\ell)^2} \right] \begin{vmatrix} x_{i'}^{\ell'} \\ x_{i'-1}^{\ell'} \end{vmatrix} \begin{vmatrix} y_{j'}^{\ell'} \\ y_{j'-1}^{\ell'} \end{vmatrix}. \quad (24)$$

The direct method follows that

$$F_x(x_{i-1/2}^\ell, y_{j-1/2}^\ell) = \sum_{\ell=K_C}^{K_F} \sum_{D_{i,j}^\ell \in S^\ell} \sigma_{i',j'}^{\ell'} \mathcal{F}_0(x_{i'-1/2}^{\ell'} - x_{i-1/2}^\ell, y_{j'-1/2}^{\ell'} - y_{j-1/2}^\ell) \quad (25)$$

for $(x_{i-1/2}^\ell, y_{j-1/2}^\ell) \in D_{i,j}^\ell \in S^\ell$. Moreover, the numerical complexity of the direct method is $O(n^2)$, where n is the total number of zones. When $K_C = K_F$ and $\Delta x = 1/2^{\ell-1}$, the grid discretization is uniform, and the formula

$$\begin{aligned} \mathcal{F}_0 = & -\ln \left((j' - j + \frac{1}{2}) + \sqrt{(i' - i + \frac{1}{2})^2 + (j' - j + \frac{1}{2})^2} \right) + \ln \left((j' - j - \frac{1}{2}) + \sqrt{(i' - i + \frac{1}{2})^2 + (j' - j - \frac{1}{2})^2} \right) \\ & + \ln \left((j' - j + \frac{1}{2}) + \sqrt{(i' - i - \frac{1}{2})^2 + (j' - j + \frac{1}{2})^2} \right) + \ln \left((j' - j - \frac{1}{2}) + \sqrt{(i' - i - \frac{1}{2})^2 + (j' - j - \frac{1}{2})^2} \right) \\ \equiv & \mathcal{F}_{0,i'-i,j'-j} \end{aligned} \quad (26)$$

Furthermore, the x -force is rewritten to

$$F_x(x_{i-1/2}^\ell, y_{j-1/2}^\ell) = \sum_{i'=1}^N \sum_{j'=1}^N \sigma_{i',j'}^\ell \mathcal{F}_{0,i'-i,j'-j} \quad (27)$$

The above formula is in the convolution form, and it is known to have a fast computation with the help of FFT. The numerical complexity is also reduced to $O(N^2 \log_2^2 N)$ from $O(N^4)$, where N is the number of zones in one dimension. However, the usage of FFT requires a homogeneous structure and a convolution form.

We advance to decompose the domain into subdomains that merit the criteria for employing FFT whenever $K_C \neq K_F$. In the following, we assume that we compute the force at D^ℓ and the surface density at $D^{\ell'}$. We consider two cases: (i) $\ell \geq \ell'$ and (ii) $\ell \leq \ell'$. Let us denote $W = 2^{|\ell-\ell'|}$.

Case (i) $\ell \geq \ell'$. The force in the finer grids induced from the surface density at the coarser grids $D^{\ell'}$ is given by

$$F_x(x_{i-1/2}^\ell, y_{j-1/2}^\ell) = \sum_{i'=1}^{2^{\ell'}} \sum_{j'=1}^{2^{\ell'}} \sigma_{i',j'}^{\ell'} \mathcal{F}_0(x_{i'-1/2}^{\ell'} - x_{i-1/2}^\ell, y_{j'-1/2}^{\ell'} - y_{j-1/2}^\ell). \quad (28)$$

Since $\ell \geq \ell'$, $x_{i'-1/2}^{\ell'} - x_{i-1/2}^\ell = 2^{-\ell}(W(i' - 1/2) - i + 1/2)$ and $y_{j'-1/2}^{\ell'} - y_{j-1/2}^\ell = 2^{-\ell}(W(j' - 1/2) - j + 1/2)$, we rewrite i as $W\tilde{i} + m_i$ and j as $W\tilde{j} + m_j$, where $m_i, m_j = 0, 1, 2, \dots, W-1$ and $\tilde{i}, \tilde{j} = 1, 2, \dots, 2^{\ell'}$. For each fixed m_i and m_j , the force is

$$F_x(x_{W\tilde{i}+m_i-1/2}^\ell, y_{W\tilde{j}+m_j-1/2}^\ell) = \sum_{i'=1}^{2^{\ell'}} \sum_{j'=1}^{2^{\ell'}} \sigma_{i',j'}^{\ell'} \mathcal{F}_0(2^{-\ell}(W(i' - \tilde{i}) - (W-1)/2 - m_i), 2^{-\ell}(W(j' - \tilde{j}) - (W-1)/2 - m_j)). \quad (29)$$

The force calculated at the finer grid is now divided into $W \times W$ sub-regions. The formula above can rewrite this procedure in the convolution form.

Case (ii) $\ell \leq \ell'$. The force in the coarser grids induced from the surface density at the finer grids $D^{\ell'}$ is given by

$$F_x(x_{i-1/2}^\ell, y_{j-1/2}^\ell) = \sum_{i'=1}^{2^{\ell'}} \sum_{j'=1}^{2^{\ell'}} \sigma_{i',j'}^{\ell'} \mathcal{F}_0(x_{i'-1/2}^{\ell'} - x_{i-1/2}^\ell, y_{j'-1/2}^{\ell'} - y_{j-1/2}^\ell). \quad (30)$$

Since $\ell \leq \ell'$, $x_{i'-1/2}^{\ell'} - x_{i-1/2}^\ell = 2^{-\ell'}(i' - 1/2 - W(i - 1/2))$ and $y_{j'-1/2}^{\ell'} - y_{j-1/2}^\ell = 2^{-\ell'}(j' - 1/2 - W(j - 1/2))$, we rewrite i' as $W\tilde{i}' + m_{i'}$ and j' as $W\tilde{j}' + m_{j'}$, where $m_{i'}, m_{j'} = 0, 1, 2, \dots, W-1$. For each fixed $m_{i'}$ and $m_{j'}$, the force is

$$\begin{aligned} F_x(x_{i-1/2}^\ell, y_{j-1/2}^\ell) = & \sum_{m_{i'}=0}^{W-1} \sum_{m_{j'}=0}^{W-1} \left[\sum_{\tilde{i}'=1}^{2^{\ell'}} \sum_{\tilde{j}'=1}^{2^{\ell'}} \sigma_{W\tilde{i}'+m_{i'}, W\tilde{j}'+m_{j'}}^{\ell'} \times \right. \\ & \left. \mathcal{F}_0(2^{-\ell'}(W(\tilde{i}' - i) - (W-1)/2 - m_{i'}), 2^{-\ell'}(W(\tilde{j}' - j) - (W-1)/2 - m_{j'})) \right]. \end{aligned} \quad (31)$$

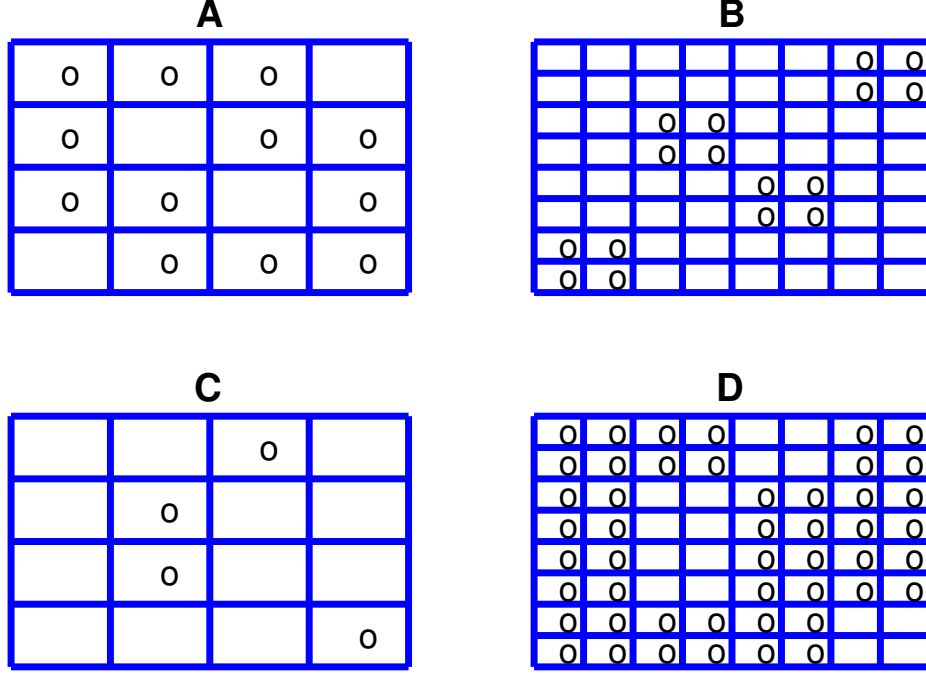


Figure 1. Schematics of a two-level refinement example. The discretization B is the mesh refinement of the discretization A, and the discretization D is the mesh refinement of the discretization C. The positions of 'o' are the places for prescribed surface density and evaluation of forces. The empty zones can be ignored during the calculation.

We estimate the numerical complexity of this new approach as follows. For a fixed ℓ , the numerical complexity for the force calculation is $O(W^2 * (2^{\ell'} \log_2 2^{\ell'})^2) \simeq O(2^{\ell} \log_2 2^{\ell})^2$ for both cases. More precisely, it only relies on the decomposition of the force or the surface density for cases (i) and (ii), respectively. The total computational time is the sum of the computational time of all levels, $O(\sum_{\ell=K_C}^{K_F} (2^{\ell} \log_2 2^{\ell})^2)$, which can be estimated to be $O((2^{K_F} \log_2 2^{K_F})^2)$. In reality, for given ℓ and ℓ' , a hybrid approach is to adopt the DnBC or FFT, depending on the estimation of computational time: $CC(DnBC) = \#(S^{\ell}) \times \#(S^{\ell'})$ and $CC(FFT) = (2^{\max\{\ell, \ell'\}}) \times \log_2(\max\{\ell, \ell'\})^2$, where $\#(S)$ is the number of zones of S . We apply the following condition for each case (ℓ, ℓ') if

$$CC(DnBC) < CC(FFT) \quad (32)$$

the direct method is adopted; otherwise, the FFT is used.

We note that the new approach can extend the computational sub-regions onto the whole domain. Some calculations of zones, which are not EN , are redundant. Therefore, we can estimate the computation time using whichever is less between the extended domain and the direct approach. That is the reason why this approach is a hybrid method. Let us perform a simple example as in Figure 2 to demonstrate the choices of DnBC or FFT. The discretization B is the mesh refinement of the discretization A, and the discretization D is the mesh refinement of the discretization C. The positions of 'o' are the places for the prescribed surface density and evaluation of forces. The empty zones can be ignored during the calculation. Table 2 shows the numerical complexity of DnBC and FFT for the situations of refinement (A,B) and (C,D) in Figure 1. The numerical complexity only concerns theoretical estimations for description of insight of the proposed methodology. $F_{\Omega} S_R$ means to calculate force at the domain Ω and the surface density given at the domain R , where $\Omega, R = A, B, C, D$. For (A,B), the method marked by '*' is used to calculate the smallest complexity, $64 + 192 + 192 + 256 = 704$. Similarly, for (C,D), the smallest complexity is $16 + 192 + 192 + 576 = 976$.

5. NUMERICAL DEMONSTRATION

We carry out numerical examples using the hybrid approach, adopting the same setups, the refinement pattern and the cases $AMR(L, D)$ for $L = 2, 3$ and $D = D_2$ or $D_{2,2}$ as in Tseng et al. (2019). Our main focus is to measure the acceleration of the force calculation, while the potential calculations are known to remain similar. We verify the accuracy of the calculation shown in Table 4 by comparing the case, $AMR(3, D_2)$ with ILR refinement in Table 3 of Tseng et al. (2019) before the acceleration study.

Type	DnBC	FFT
$F_A S_A$	$12 \times 12 = 144$	$*(4 \log_2 4)^2 = 64$
$F_A S_B$	$*12 \times 16 = 192$	$4 \times (4 \log_2 4)^2 = 256$
$F_B S_A$	$*16 \times 12 = 192$	$4 \times (4 \log_2 4)^2 = 256$
$F_B S_B$	$*16 \times 16 = 256$	$(8 \times \log_2 8)^2 = 576$
$F_C S_C$	$*4 \times 4 = 16$	$(4 \log_2 4)^2 = 64$
$F_C S_D$	$*4 \times 48 = 192$	$4 \times (4 \log_2 4)^2 = 256$
$F_D S_C$	$*48 \times 4 = 192$	$4 \times (4 \log_2 4)^2 = 256$
$F_D S_D$	$48 \times 48 = 2304$	$*(8 \times \log_2 8)^2 = 576$

Table 2. This table shows the complexity of DnBC and FFT for the situations of refinement (A,B) and (C,D) in Figure 2. $F_\Omega S_R$ means to calculate force at the domain Ω and the surface density given at the domain R , where $\Omega, R = A, B, C, D$. The smaller complexity for each calculation is indicated with an asterisk.

We simulate the case for the number $N_k = 2^K$ of grid zones in each direction for $K = 7, 8, \dots, 11$. The errors are measured by L_1 , L_2 , and L_∞ , given by

$$|u|_{L_1} \equiv \sum_i |u_i| \Delta A_i, \quad |u|_{L_2} \equiv \left(\sum_i (u_i)^2 \Delta A_i \right)^{1/2}, \quad |u|_{L_\infty} \equiv \max_i |u_i|,$$

where ΔA_i is the area of the zones. The errors between the numerical and analytic solutions are calculated for the force F_x in the x -direction, and the (radial) force F_R R -direction in different norms, and they are denoted by $L_{\alpha,\beta}$ with $\alpha = x, R$ and $\beta = 1, 2, \infty$. The orders of accuracy $O_{\alpha,\beta}$ at the bottom portion of Table 4 (Appendix) is defined by $\log_2(L_{\alpha,\beta}(N_k)/L_{\alpha,\beta}(N_{k+1}))$, where $\alpha = x, R$ and β equals 1, 2, ∞ .

Tables 5 and 6 show the resulting computation time for the ten simulations with a 1% standard deviation using the same computational platforms described in Section 2. In Table 5, the $EL/(N_k^2)$ ratios for $N_k = 2^k$, $k = 7, 8, \dots, 11$ and D_2 using ILR are fixed at 1.12 and 1.36 for $L = 2$ and $L = 3$, respectively. The EL is the number of the leaf zones interacting with each other by gravity for the AMR. The N_k^2 is the zone number of the uniform grid. For $D_{2,2}$ the ratios are 1.19 and 1.57 for $L = 2$ and $L = 3$, respectively. The computational time increases exponentially with N_k . The behavior is similar for VLR, as reported in Table 5.

For the computational time of the hybrid method, there are two refinement strategies of ILR and VLR for two disks, D_2 and $D_{2,2}$. Table 5 shows the results for ILR, and Table 6 shows the results for VLR (see Appendix). In the last column, CUDA/HM means the ratio, $\text{CT}(\text{CUDA})/\text{CT}(\text{HM})$, where $\text{CT}(\text{CUDA})$ is the computational time by CUDA and $\text{CT}(\text{HM})$ is the computational time by the hybrid method. The ratio of the computational times HM/UG is given by the results from the HM and the uniform grid (UG), respectively. In Table 4, the results show that CUDA/HM are 26.6 and 7.79 for $\text{AMR}(2, D_2)$ for 2048 and 1024 zones in one dimension, respectively. For $\text{AMR}(3, D_2)$, the ratio is 5.3 and 1.63 for 2048 and 1024, respectively. Moreover, for $\text{AMR}(2, D_{2,2})$ for 2048 and 1024 zones in one dimension are 31.1 and 12.3, respectively. The ratios for $\text{AMR}(3, D_{2,2})$ are 7.3 and 3.06 for 2048 and 1024 zones. The results for VLR, as shown in Table 5, are similar to those for ILR. The process of refinement to the sub-regions and the refined zones depends on the location.

We adopt a new "refined-zone" approach for the computation mentioned above. The approach is to refine the zones which have the larger densities with a ratio r during the level increase. A two-arm spiral (TAS) similar to a spiral galaxy is used for probing the behavior of computation time, which has a surface density $\sigma = \sigma(r, \theta)$ in the polar coordinate given by

$$\frac{10}{2+r} (\exp(-2(r - \theta/3)) + \exp(-2(r - \text{mod}(\theta - \pi)/3))) \quad (33)$$

in polar coordinates, whose profile is shown in Fig. 2. The surface density decays exponentially from high density region at the center. The computational domain is $[-1, 1] \times [-1, 1]$. The results of the two calculations for the level refinements $L = 2$ and $L = 3$ are shown in the top and bottom tables in Table 6. The first column (NC, NR) means that $NC^2 + NR^2$ is the number of zones on the coarsest level and the refinement approach uses the larger surface density with the ratio 10% during level increase. **More precisely, the cells of the large top 10% of surface density are**

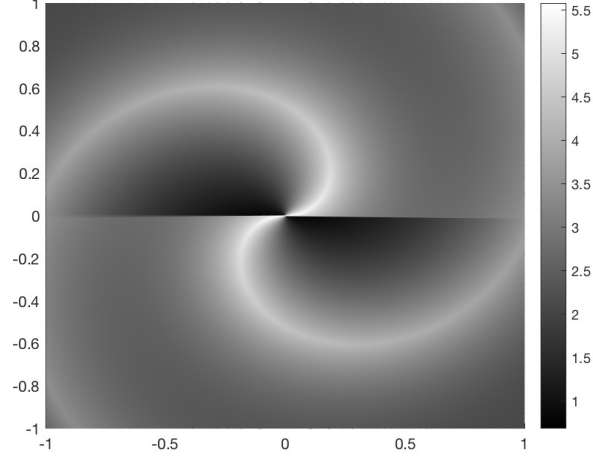


Figure 2. A two-arm spiral (TAS) example with a surface density $\frac{10}{2+\tau}(\exp(-2(r-\theta/3)) + \exp(-2(r-\text{mod}(\theta-\pi)/3)))$ in polar coordinates on $[-1, 1] \times [-1, 1]$ is used to investigate the computational time of proposed hybrid method, and is shown to be nearly linear.

used to be refined among all cells at the same level. The values of CT are the mean of 10 computations for each case, and the corresponding standard deviation is in the column STDEV. The STDEV/CT is the ratio of STDEV over CT.

We investigated the two-arm spiral with a surface density close to the spiral galaxies to analyze the curves of EL vs CT in Tables 5, 6, and 3 by log-log plots. In Figure 3, the six curves in the left panel are for AMR(1, D_2), AMR(2, D_2), AMR(3, D_2), AMR(1, $D_{2,2}$), AMR(2, $D_{2,2}$), and AMR(3, $D_{2,2}$) marked by '▽', '○', '✱', '△', '◇', and '+', respectively; similarly, there are six curves in the middle panel of Table 6. There are two curves in the right panel for $L = 2$ and $L = 3$ of Table 3. The results show that the computational time of this newly proposed "hybrid" method is nearly linear whenever the number of EL zones increases.

6. CONCLUSION AND DISCUSSION

We have deployed a hybrid method that combines a direct approach with FFT-based homogeneous decomposition, significantly reducing computational time. Comparisons with GPU computations reveal that our hybrid approach (HM) outperforms the direct method (DnBC), exhibiting acceleration ratios ranging from 3.56 to 204 for 2048^2 zones within disks $AMR(L, D)$, where $L = 2, 3$ and D_2 or $D_{2,2}$.

Analysis from Tables 4 and 5, portraying $CT(HM)/CT(UG)$, elucidates the impact of adaptive mesh refinement of HM on computational time relative to the uniform grids (UG). The findings indicate a nearly constant growth in computational time, around a factor of 4, corresponding to the expansion of grid zones with the increase in parameter K . Our novel method exhibits an improvement of approximately 11 or 15.8, surpassing the results presented in Tseng et al. (2019). The values CT for $N_k = 1024$ in Table 9 of Tseng et al. (2019) are $(5.99 \times 10^2, 7.94 \times 10^2, 2.01 \times 10^3, 2.53 \times 10^3)$ and the corresponding values are $(8.63 \times 10^1, 6.55 \times 10^2, 8.88 \times 10^1, 6.45 \times 10^2)$ in Table 4, respectively. Therefore, we conclude that the numerical examples are viable for 1024×1024 zones in about 3 days, a 70% reduction from 10 days in Tseng et al. (2019).

The condition (32) is analytic. It may depend on the implemented coding technique, computer equipment and simulation software, and this condition can be modified into

$$CT(DnBC) < r_f CT(FFT) \quad (34)$$

where r_f is a ratio. In reality, the ratio r_f may be larger than 1 because implementing FFT will involve kernel computation and depends on kernel storage treatment in computer memory. The number r_f in (34) will also depend on the level.

TAS- (NC, NR)	$L = 1$	$L = 2$	$EL(\text{leaf})$	$EL/(N_k^2)$	CT(sec)	$CT(N_k)/CT(N_{k-1})$	STDEV	STDEV/CT
(256, 0)	65536	92428	85705	1.31E+00	2.88E+00		4.82E-02	1.68E-02
(256, 16)	73984	104512	96880	1.31E+00	3.81E+00	1.33E+00	5.71E-02	1.50E-02
(256, 32)	82944	116708	108267	1.31E+00	3.49E+00	9.15E-01	9.74E-02	2.79E-02
(256, 64)	102400	144908	134281	1.31E+00	4.45E+00	1.27E+00	7.32E-02	1.65E-02
(256, 128)	147456	206688	191880	1.30E+00	6.44E+00	1.45E+00	9.08E-02	1.41E-02
(512, 0)	262144	371736	344338	1.31E+00	1.41E+01	2.18E+00	1.36E-01	9.67E-03
(512, 16)	278784	393076	364503	1.31E+00	1.34E+01	9.50E-01	2.84E-01	2.12E-02
(512, 32)	295936	421028	389755	1.32E+00	1.69E+01	1.26E+00	2.58E-01	1.53E-02
(512, 64)	331776	469652	435183	1.31E+00	1.54E+01	9.10E-01	2.31E-01	1.50E-02
(512, 128)	409600	580264	537598	1.31E+00	2.05E+01	1.34E+00	4.88E-01	2.38E-02
(512, 256)	589824	833048	772242	1.31E+00	3.12E+01	1.52E+00	1.35E-01	4.34E-03
(1024, 0)	1048576	1487080	1377454	1.31E+00	7.55E+01	2.42E+00	2.69E-01	3.55E-03
(1024, 16)	1081600	1536708	1422931	1.32E+00	5.92E+01	7.84E-01	4.52E-01	7.62E-03
(1024, 32)	1115136	1580216	1463946	1.31E+00	6.05E+01	1.02E+00	3.23E-01	5.33E-03
(1024, 64)	1183744	1676260	1553131	1.31E+00	7.77E+01	1.28E+00	2.30E-01	2.96E-03
(1024, 128)	1327104	1879300	1741251	1.31E+00	7.45E+01	9.59E-01	2.84E-01	3.82E-03
(1024, 256)	1638400	2328564	2156023	1.32E+00	9.86E+01	1.32E+00	3.99E-01	4.05E-03
(1024, 512)	2359296	3345412	3098883	1.31E+00	1.62E+02	1.64E+00	4.28E-01	2.65E-03
TAS- (NC, NR)	$L = 1$	$L = 3$	$EL(\text{leaf})$	$EL/(N_k^2)$	CT(sec)	$CT(N_k)/CT(N_{k-1})$	STDEV	STDEV/CT
(256, 0)	65536	198368	165160	2.52E+00	2.21E+01		2.76E-01	1.25E-02
(256, 16)	73984	225056	187288	2.53E+00	2.87E+01	1.30E+00	2.10E-01	7.34E-03
(256, 32)	82944	250004	208239	2.51E+00	2.63E+01	9.17E-01	2.74E-01	1.04E-02
(256, 64)	102400	312948	260311	2.54E+00	3.42E+01	1.30E+00	2.26E-01	6.60E-03
(256, 128)	147456	441208	367770	2.49E+00	5.02E+01	1.47E+00	1.34E-01	2.67E-03
(512, 0)	262144	806792	670630	2.56E+00	1.14E+02	2.27E+00	3.16E-01	2.77E-03
(512, 16)	278784	846884	704859	2.53E+00	9.96E+01	8.72E-01	3.41E-01	3.42E-03
(512, 32)	295936	918100	762559	2.58E+00	1.29E+02	1.30E+00	3.44E-01	2.65E-03
(512, 64)	331776	1017352	845958	2.55E+00	1.18E+02	9.15E-01	9.08E-01	7.67E-03
(512, 128)	409600	1258912	1046584	2.56E+00	1.59E+02	1.34E+00	2.97E-01	1.86E-03
(512, 256)	589824	1801352	1498470	2.54E+00	2.47E+02	1.55E+00	2.24E+00	9.05E-03
(1024, 0)	1048576	3234616	2688106	2.56E+00	5.72E+02	2.31E+00	3.76E-01	6.58E-04
(1024, 16)	1081600	3350500	2783275	2.57E+00	4.33E+02	7.57E-01	6.25E-01	1.44E-03
(1024, 32)	1115136	3433872	2854188	2.56E+00	4.51E+02	1.04E+00	3.38E-01	7.49E-04
(1024, 64)	1183744	3639152	3025300	2.56E+00	6.06E+02	1.34E+00	5.10E-01	8.41E-04
(1024, 128)	1327104	4080932	3392475	2.56E+00	5.91E+02	9.76E-01	7.77E-01	1.31E-03
(1024, 256)	1638400	5081164	4220473	2.58E+00	7.78E+02	1.32E+00	8.59E-01	1.10E-03
(1024, 512)	2359296	7280100	6049899	2.56E+00	1.29E+03	1.66E+00	1.60E+00	1.24E-03

Table 3. The computational time for the spiral potential. (NC, NR) , where NC means the coarsest number of grid zones and finer number of grid zones for the level $L = 2$ and 3 simulations by CUDA. N_k is the number of zones in one dimension. The numbers in $L = 2$ or $L = 3$ are the total numbers of zones, and $EL(\text{leaf})$ is the corresponding number of leaf zones. CT stands for the computational time. The EL/N_k^2 is the ratio of the number of leaf zones divided by N_k^2 . The ratios $CT(N_k)/CT(N_{k-1})$ shows that the computational time exponentially grows with N_k for $k = 7, 8, 9, 10, 11$. In the last column, CUDA/HM means the ratio, $CT(\text{CUDA})/CT(\text{HM})$, where $CT(\text{CUDA})$ is the computational time by CUDA and $CT(\text{HM})$ is the computational time by the hybrid method. There are 10 calculations performed for each case, and the STDEV is the corresponding standard deviation.

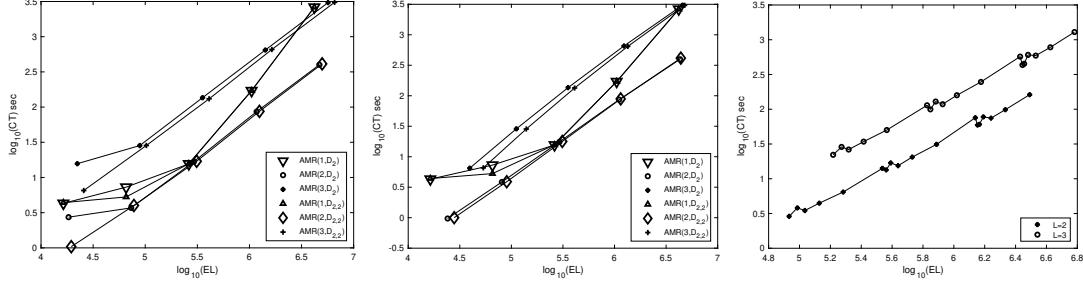


Figure 3. These curves in each subplot are the number of leaf zones vs the computation time, EL vs CT, in Table 5,6,3 in log-log plot. There six curves in the left subplot are for AMR(1, D_2), AMR(2, D_2), AMR(3, D_2), AMR(1, $D_{2,2}$), AMR(2, $D_{2,2}$) and AMR(3, $D_{2,2}$) marked by '▽', '○', '•', '△', '◇', and '+' respectively. Similarly, for the curves in the middle subplot in Table 6 for the cases with ILR. There are two curves in the right subplot for $L = 2$ and $L = 3$ in Table 3 for the cases with VLR. The results show that the computational time is nearly linear whenever the numbers of EL zones increase.

The authors acknowledge grant support for the CompAS group from the Institute of Astronomy and Astrophysics, Academia Sinica (ASIAA). C.C. Yen acknowledges grant support from the National Science and Technology Council (NSTC) under 112-2115-M-030-002- and 113-2115-M-030-002-. H.S. acknowledges support from the National Science and Technology Council (NSTC) in Taiwan through grant 113-2112-M-001-008- and an Academia Sinica grant AS-IAIA-114-M01. The authors acknowledge access to high-performance facilities in ASIAA and thank the National Center for High-Performance Computing (NCHC) of the National Applied Research Laboratories (NARLabs) in Taiwan for providing computational and storage resources. This work utilized tools developed and maintained by the ASIAA CompAS group.

APPENDIX

In this Appendix, we list additional tables to help illustrate the computations for the **HM** because some data have been shown in Tseng et al. (2019). We confirm our results in Table 4, which shows the errors and order of accuracy for AMR(3, D_2) with $L = 3$ and $\alpha = 0.25$. We also include results from Tables 8 and 9 in Tseng et al. (2019) as Tables 5 and 6, below, to compare with results obtained in this work. The rightmost columns of CUDA/**HM** in both Tables are calculated using the proposed hybrid method as discussed in Section 5.

AMR(3, D_2)- N_k	$L_{x,1}$	$L_{x,2}$	$L_{x,\infty}$	$L_{R,1}$	$L_{R,2}$	$L_{R,\infty}$
128	5.32E-4	5.50E-4	1.14E-3	8.29E-4	7.76E-4	1.17E-3
256	1.34E-4	1.40E-4	3.62E-4	2.10E-4	1.98E-4	3.66E-4
512	3.38E-5	3.53E-5	1.30E-4	5.30E-5	4.98E-5	1.41E-4
1024	8.48E-6	8.86E-6	5.06E-5	1.33E-5	1.25E-5	5.09E-5
2048	2.12E-6	2.22E-6	1.84E-5	3.33E-6	3.14E-6	1.84E-5
AMR(3, D_2)- N_k/N_{k+1}	$O_{x,1}$	$O_{x,2}$	$O_{x,\infty}$	$O_{R,1}$	$O_{R,2}$	$O_{R,\infty}$
128/256	1.99	1.97	1.65	1.98	1.97	1.68
256/512	1.99	1.99	1.47	1.99	1.99	1.37
512/1024	2.00	1.99	1.36	1.99	1.99	1.47
1024/2048	2.00	2.00	1.46	2.00	2.00	1.47

Table 4. The Simulation of AMR(3, D_2) for D_2 Disk with $L = 3$ and $\alpha = 0.25$

REFERENCES

- Bandopadhyay, S., and Shang, H. 2022, ApJS 263, 32
- Deriaz, E. 2023, JCoPh, 480, 112012

AMR(2, D_2)- N_k	$L = 1; N_k^2$	$L = 2$	$EL(\text{leaf})$	$EL/(N_k^2)$	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	19056	18388	1.12E+0	2.73E+0	—	2.27E+1	1.67E+0
256	65536	75792	73228	1.12E+0	3.70E+0	1.36E+0	1.30E+1	1.97E+0
512	262144	303344	293044	1.12E+0	1.68E+1	4.53E+0	1.33E+1	2.77E+0
1024	1048576	1213376	1172176	1.12E+0	8.61E+1	5.13E+0	1.30E+1	7.79E+0
2048	4194304	4853408	4688632	1.12E+0	3.99E+2	4.63E+0	1.29E+1	2.66E+1
AMR(3, D_2)- N_k	$L = 1; N_k^2$	$L = 3$	$EL(\text{leaf})$	EL/N_k^2	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	24272	22300	1.36E+0	1.57E+1	—	1.31E+2	2.94E-1
256	65536	96256	88576	1.35E+0	2.84E+1	1.80E+0	9.15E+1	3.05E-1
512	262144	385712	354820	1.35E+0	1.36E+2	4.78E+0	9.90E+1	5.17E-1
1024	1048576	1543040	1419424	1.35E+0	6.46E+2	4.76E+0	9.51E+1	1.63E+0
2048	4194304	6171680	5677336	1.35E+0	3.05E+3	4.73E+0	9.58E+1	5.30E+0
AMR(2, $D_{2,2}$)- N_k	$L = 1; N_k^2$	$L = 2$	$EL(\text{leaf})$	$EL/(N_k^2)$	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	20544	19504	1.19E+0	1.04E+0	—	9.45E+0	4.41E+0
256	65536	81984	77872	1.19E+0	4.00E+0	3.85E+0	1.29E+1	2.05E+0
512	262144	327904	311464	1.19E+0	1.79E+1	4.47E+0	1.29E+1	3.75E+0
1024	1048576	1312192	1246288	1.19E+0	8.69E+1	4.86E+0	1.29E+1	1.23E+1
2048	4194304	5248128	4984672	1.19E+0	4.11E+2	4.73E+0	1.40E+1	3.11E+1
AMR(3, $D_{2,2}$)- N_k	$L = 1; N_k^2$	$L = 3$	$EL(\text{leaf})$	EL/N_k^2	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	28800	25696	1.57E+0	6.53E+0	—	5.94E+1	7.46E-1
256	65536	114848	102520	1.56E+0	2.85E+1	4.37E+0	9.21E+1	4.17E-1
512	262144	459648	410272	1.57E+0	1.31E+2	4.61E+0	9.46E+1	9.66E-1
1024	1048576	1838720	1641184	1.57E+0	6.58E+2	4.86E+0	9.73E+1	3.06E+0
2048	4194304	7356544	6565984	1.57E+0	3.08E+3	4.69E+0	1.05E+2	7.30E+0

Table 5. The computational time for AMR(L, D_2) and AMR($L, D_{2,2}$) for the level $L = 2$ and 3 simulations with ILR by the proposed hybrid method (HM) and CUDA. The N_k is the number of zones in one dimension. The numbers in $L = 2$ or $L = 3$ are the total numbers of zones, $EL(\text{leaf})$ is the corresponding number of leaf zones. CT(HM) means the computational time. The EL/N_k^2 is the ratio of the number of leaf zones divided by N_k^2 . The ratios CT(N_k)/CT(N_{k-1}) show that the computational time exponentially grows with N_k for $k = 7, 8, 9, 10, 11$. In the last column, CUDA/HM means the ratio, CT(CUDA)/CT(HM), where CT(CUDA) is the computational time by CUDA and CT(HM) is the computational time by the hybrid method. The results show that CUDA/HM are 26.6 and 7.79 for AMR(2, D_2) for 2048 and 1024 zones in one dimension, respectively. For AMR(3, D_2), the ratio is 5.3 and 1.63 for 2048 and 1024 zones, respectively. Moreover, for AMR(2, $D_{2,2}$) for 2048 and 1024 zones in one dimension are 31.1 and 12.3, respectively. The ratios for AMR(3, $D_{2,2}$) are 7.3 and 3.06 for 2048 and 1024 zones.

James, 1977, JCoPh, 25, 71

Kim, W.-T., Seo, W.-Y., & Kim, Y. 2012, ApJ, 758, 14

Lee, W.-K. 2014, ApJ, 792, 122

Lee, W.-K., & Shu, F. H. 2012, ApJ, 756, 45

Lin, L.-H., Yuan, C., & Buta, R. 2008, ApJ, 684, 1048

Lin, L.-H., Wang, H.-H., Hsieh, P.-Y., et al. 2013, ApJ, 771, 8

Maeda, R., Inoue, T., Inutsuka, S.I. 2024, MNRAS, 527, 471

Mandal A., Mukherjee D. and Mignone A. 2023, ApJS, 260, 40

Shulz, E., 2009, ApJ, 693, 1310.

Tomida, K. and Stone, J.M. 2023, ApJS, 266, 7

Tseng, Y.-H., Hsien, H., Yen, C.-C., 2019, ApJS, 244, 26

Wang, H.-H., Taam, R. E., Yen, C.-C., 2016, ApJS, 224, 16

Yen, C.-C., Taam, R. E., Yeh, K. H.-C., & Jea, K. C. 2012, JCoPh, 231, 8246

AMR(2, D_2)- N_k	$L = 1; N_k^2$	$L = 2$	$EL(\text{leaf})$	$EL/(N_k^2)$	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	26672	24100	1.47E+0	9.70E-1	—	8.08E+0	4.75E+0
256	65536	86032	80908	1.23E+0	3.85E+0	3.97E+0	1.24E+1	2.10E+0
512	262144	303344	293044	1.18E+0	1.76E+1	4.58E+0	1.29E+1	2.63E+0
1024	1048576	1130944	1110352	1.06E+0	8.63E+1	4.90E+0	1.27E+1	6.94E+0
2048	4194304	4358832	4317700	1.03E+0	3.97E+2	4.59E+0	1.24E+1	2.27E+1
AMR(3, D_2)- N_k	$L = 1; N_k^2$	$L = 3$	$EL(\text{leaf})$	EL/N_k^2	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	47168	39472	2.41E+0	6.49E+0	—	5.41E+1	8.26E-1
256	65536	127232	111808	1.71E+0	2.87E+1	4.42E+0	9.26E+1	4.36E-1
512	262144	385712	354820	1.35E+0	1.36E+2	4.73E+0	9.91E+1	5.14E-1
1024	1048576	1295472	1233748	1.18E+0	6.55E+2	4.82E+0	9.64E+1	1.21E+0
2048	4194304	4688352	4564840	1.09E+0	3.06E+3	4.68E+0	9.61E+1	3.56E+0
AMR(2, $D_{2,2}$)- N_k	$L = 1; N_k^2$	$L = 2$	$EL(\text{leaf})$	$EL/(N_k^2)$	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	32928	28792	1.76E+0	1.00E+0	—	9.09E+0	4.97E+0
256	65536	98400	90184	1.37E+0	3.89E+0	3.89E+0	1.25E+1	2.38E+0
512	262144	327904	311464	1.19E+0	1.78E+1	4.57E+0	1.28E+1	3.96E+0
1024	1048576	1180320	1147384	1.09E+0	8.88E+1	5.00E+0	1.31E+1	2.26E+1
2048	4194304	4458080	4392136	1.05E+0	4.14E+2	4.66E+0	1.41E+1	2.04E+2
AMR(3, $D_{2,2}$)- N_k	$L = 1; N_k^2$	$L = 3$	$EL(\text{leaf})$	EL/N_k^2	CT(HM)(sec)	CT(N_k)/CT(N_{k-1})	HM/UG	CUDA/HM
128	16384	65792	53440	3.26E+0	6.57E+0	—	5.97E+1	1.08E+0
256	65536	164160	139504	2.13E+0	2.85E+1	4.34E+0	9.21E+1	5.57E-1
512	262144	459648	410272	1.57E+0	1.34E+2	4.69E+0	9.63E+1	9.49E-1
1024	1048576	1444096	1345216	1.28E+0	6.45E+2	4.82E+0	9.54E+1	3.92E+0
2048	4194304	4985152	4787440	1.14E+0	3.04E+3	4.72E+0	1.04E+2	4.21E+1

Table 6. The computational time for AMR(L, D_2) and AMR($L, D_{2,2}$) for the level $L = 2$ and 3 simulations with VLR by the proposed hybrid method (HM) and CUDA. N_k is the number of zones in one dimension. The numbers in $L = 2$ or $L = 3$ are the total numbers of zones, and $EL(\text{leaf})$ is the corresponding number of leaf zones. CT(HM) stands for the computational time by the hybrid method. The EL/N_k^2 is the ratio of the number of leaf zones divided by N_k^2 . The ratios CT(N_k)/CT(N_{k-1}) shows that the computational time exponentially grows with N_k for $k = 7, 8, 9, 10, 11$. In the last column, CUDA/FFT means the ratio, CT(CUDA)/CT(HM), where CT(CUDA) is the computational time by CUDA and CT(HM) is the computational time by the hybrid method. The results show that CUDA/HM are 22.7 and 6.94 for AMR(2, D_2) for 2048 and 1024 zones in one dimension, respectively. For AMR(3, D_2), the ratio is 3.56 and 1.21 for 2048 and 1024, respectively.